ESCUELA POLITÉCNICA NACIONAL

ISWD652 CALIDAD DE SOFTWARE
Período 2026B

NOMBRE ESTUDIANTE (S):	Melany Alarcón, Jhostin Cadena, Franciss Cartagena, Alisson Espin, Sebastían Guerrero, Josselyn Pozo
FECHA:	10-11-2026
TEMA:	Proyecto Versión 1: Huertos Medicinales Comunitarios

DESCRIPCIÓN DE LA TAREA

1. Participar en el taller en clase de definición del proyecto con preguntas al cliente por parte del equipo.
2. Revisar detenidamente el documento de definición de proyecto.
3. Estudiar el documento de lineamientos del MSP e investigar información complementaria con el apoyo de IA, para entender el dominio del proyecto.
4. Desarrollar la versión 1 del producto de software. Los entregables de la versión 1 incluyen: historias de usuario, arquitectura y diseño del producto, listado de herramientas y tecnologías de desarrollo, código fuente, casos de prueba con resultados, producto operativo.
5. El gestor del proyecto sube un informe elaborado utilizando la plantilla de Word con el nombre Proyecto-Version1-nombreEquipo.pdf. El informe debe contener el enlace al repositorio GitHub o similar donde se encuentre la documentación v1, el código v1, y el archivo Readme del proyecto.
6. La demo práctica del producto se realizará el martes 10 de noviembre durante la clase presencial.

CONTENIDO

1. Información general del proyecto

Empresa desarrolladora: Nexo.
Cliente: Ministerio de Salud Pública del Ecuador.
Producto: aplicación web educativa y gamificada de huertos medicinales comunitarios.
Repositorio del proyecto: https://github.com/wesdell/nexo-huertos-medicinales
Versión entregada: v2.0.0. El equipo entregó primero un producto mínimo (release v1.0.0, 6 de noviembre de 2026). Después de la reunión con la cliente del 6 de noviembre y del pedido de presentar el proyecto terminado, se completó todo el alcance en la versión 2.0.0, que es la que se presenta y se describe en este informe.

1.1 Equipo y roles

Rol	Integrante
Gestor del proyecto	Sebastían Guerrero
Analista	Melany Alarcón
Diseñador	Jhostin Cadena
Programador 1	Franciss Cartagena
Programador 2	Alisson Espin

Rol	Integrante
Tester	Josselyn Pozo

2. Historias de usuario

Los requisitos del producto se describen en 52 historias de usuario, de la HU-001 a la HU-052, agrupadas en siete módulos. Cada historia tiene su prioridad, su estimación en horas, los criterios de aceptación que dicen cuándo está terminada y las tareas de implementación. Todas están escritas con la plantilla de la asignatura en docs/historias-de-usuario/historias-de-usuario.docx.

Tabla 2.1. Módulos de historias de usuario

Módulo	Historias	Qué cubre
1. Registro	HU-001 a HU-003	Acceso, cierre de sesión y datos personales.
2. Comunidades	HU-004 a HU-009	Comunidad del huerto, miembros y foro de saberes.
3. Huerto	HU-010 a HU-016	Ubicación, diseño, cerca, suelo, agua, abono, plantas, bodega y huerto virtual.
4. Actividades	HU-017 a HU-029	Calendario lunar y los 13 pasos del cultivo según los lineamientos del MSP [1].
5. Puntos	HU-030 a HU-035	Puntos, insignias, misiones, niveles, progreso y ranking.
6. Protección de datos	HU-036	Consentimiento según la LOPDP.
7. Modelo acordado con el MSP	HU-037 a HU-052	Solicitud y aprobación del huerto, voluntarios sin cuenta, catálogo por altitud, evidencias validadas, rankings, parte pública y administración.

2.1 Cambios por la reunión con el Ministerio de Salud Pública

El 6 de noviembre de 2026 otro grupo del curso se reunió con la Ing. Sandra Sánchez, del MSP, que es la misma cliente de este proyecto. El equipo revisó el resumen de esa reunión y adoptó lo que aplica a Nexo. El cambio más grande es el modelo de cuentas: ya nadie se registra por su cuenta. Una persona de la comunidad solicita su huerto, un técnico del MSP la aprueba y entonces se crea la cuenta del responsable. Los voluntarios no tienen cuenta: los registra el responsable.

Por eso se escribieron 16 historias nuevas (HU-037 a HU-052). Las historias de la primera versión que la reunión dejó sin efecto, como el auto-registro o los puntos por tipo de actividad, siguen en el documento marcadas como reemplazadas, para que se vea la trazabilidad. La Tabla 2.2 resume los acuerdos principales y cómo quedaron en el producto.

Tabla 2.2. Acuerdos de la reunión y cómo quedaron en Nexo

Acuerdo de la reunión	Cómo quedó en Nexo	HU
Roles: técnico del MSP, responsable, invitado sin cuenta y administrador técnico	Tres cuentas (responsable, técnico del MSP y administrador) y una parte pública sin cuenta	002, 049, 051
El público solicita el huerto con lo mínimo: ubicación, tipo, subtipo, nombre, cédula, correo y celular	Formulario público con mapa y GPS; la cédula se valida con el dígito verificador	037

Acuerdo de la reunión	Cómo quedó en Nexo	HU
Aprobada: usuario y contraseña inicial por correo, que debe cambiarse. Rechazada: se archiva con motivo	Aprobación con correo automático y cambio de contraseña obligatorio en el primer ingreso	038, 039
Los voluntarios no tienen cuenta; un voluntario puede estar en varios huertos	El responsable registra voluntarios, los marca inactivos y pide su nivel por experiencia	040, 041
Catálogo de plantas del MSP, filtrado por altitud; otra altitud solo advierte	Catálogo global con carga por CSV; aviso «¿Estás seguro?» si la planta no es de esa altitud	042, 043
Actividades en secuencia: el sistema muestra el paso siguiente con sus indicaciones	Los 13 pasos de los lineamientos del MSP en orden, con «cuándo» y «cómo»	044, 052
Puntos por hora trabajada, otorgados cuando el MSP valida la evidencia	Bandeja de evidencias del técnico; al aprobar se acreditan los puntos	045
Evidencia inválida: segunda oportunidad con menos puntos	Segundo intento al 50 %; un tercero ya no da puntos	045
Sin expulsiones; una cosecha perdida se justifica con una causa	Ciclo fallido con causa y justificación; el huerto sigue	046
Rankings por tipo de huerto normalizados por voluntarios y producción anual	Ranking por tipo y ranking de paquetes producidos en el año	048
Máximo 24 horas por voluntario y por día	Validación que suma las horas del día en todos los huertos	044

La reunión dejó al equipo cuatro definiciones pendientes. El equipo hizo una propuesta para cada una, la programó en el paquete de reglas compartidas con sus pruebas y la explica al público en la página «Cómo funcionan los puntos» (Tabla 2.3).

Tabla 2.3. Propuestas del equipo para los pendientes de la reunión

Pendiente	Propuesta del equipo
Fórmula del ranking	Índice = puntos validados de los últimos 12 meses ÷ √(voluntarios activos + 1). La raíz equilibra: no favorece al huerto de una sola persona ni a los grupos grandes. En empate gana quien llegó primero a su puntaje.
Requisitos de nivel	Aprendiz al empezar. Oficial con 40 horas validadas en 4 tipos de actividad. Maestro con 150 horas, 8 tipos de actividad y un ciclo completo hasta el empaquetado. El nivel nunca baja.
Puntos	10 puntos por hora y por voluntario, y 10 % más si la actividad siguió el calendario lunar y los horarios de los saberes. Primer intento de evidencia al 100 %, segundo al 50 %.
Causas de fracaso de una cosecha	Helada, sequía, exceso de lluvia, plaga, enfermedad de la planta, daño por animales, descuido, robo u otra causa con justificación.

3. Arquitectura y diseño del producto

3.1 Arquitectura del producto

La aplicación usa una arquitectura cliente-servidor desacoplada: una aplicación de página única en React que consume una API REST en NestJS, con PostgreSQL como base de datos. A los tres contenedores de la primera versión se sumó un cuarto, Mailpit, un servidor de correo de prueba que recibe los correos con las contraseñas temporales y los muestra en una bandeja web. Las fotos y videos de evidencia se guardan en un volumen de Docker. Las reglas del dominio viven en un paquete compartido que usan el frontend y el backend.

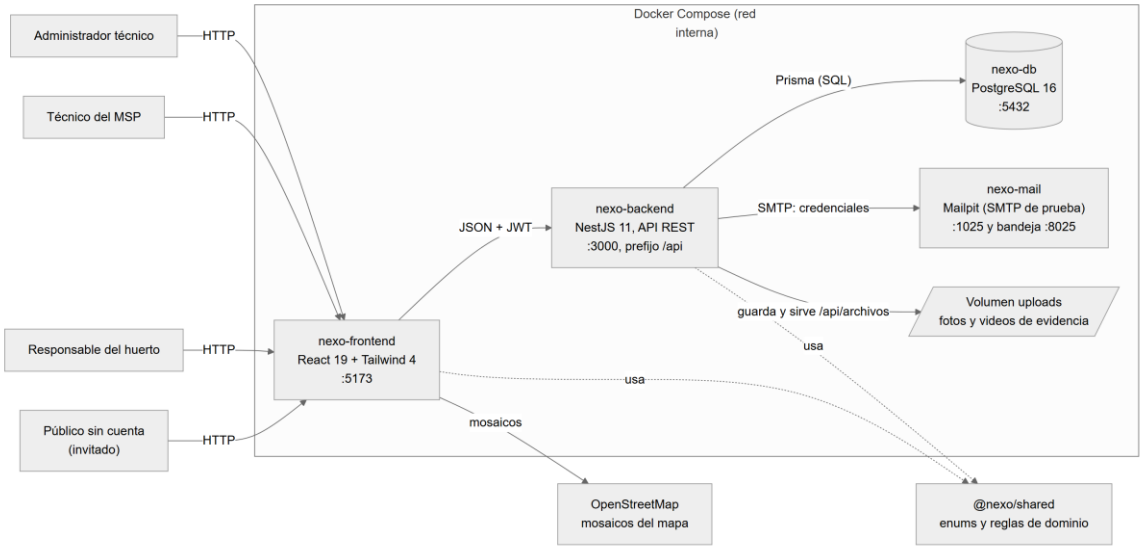


Figura 3.1. Diagrama de contenedores

El modelo de datos es relacional y está definido en Prisma. Las entidades principales son Usuario, SolicitudHuerto, Huerto, Voluntario, Planta, Ciclo, Actividad, Participación y MovimientoPuntos. Cuatro decisiones lo distinguen. Los voluntarios no son usuarios: son registros del huerto y pueden estar en varios huertos a la vez. Las plantas forman un catálogo único del MSP y cada huerto elige las suyas. Cada actividad pertenece a un ciclo de cultivo y guarda el estado de su evidencia y el número de intento. Y los puntos se guardan como movimientos por voluntario, que sirven de registro de auditoría; los totales del huerto y del voluntario se actualizan en la misma transacción.

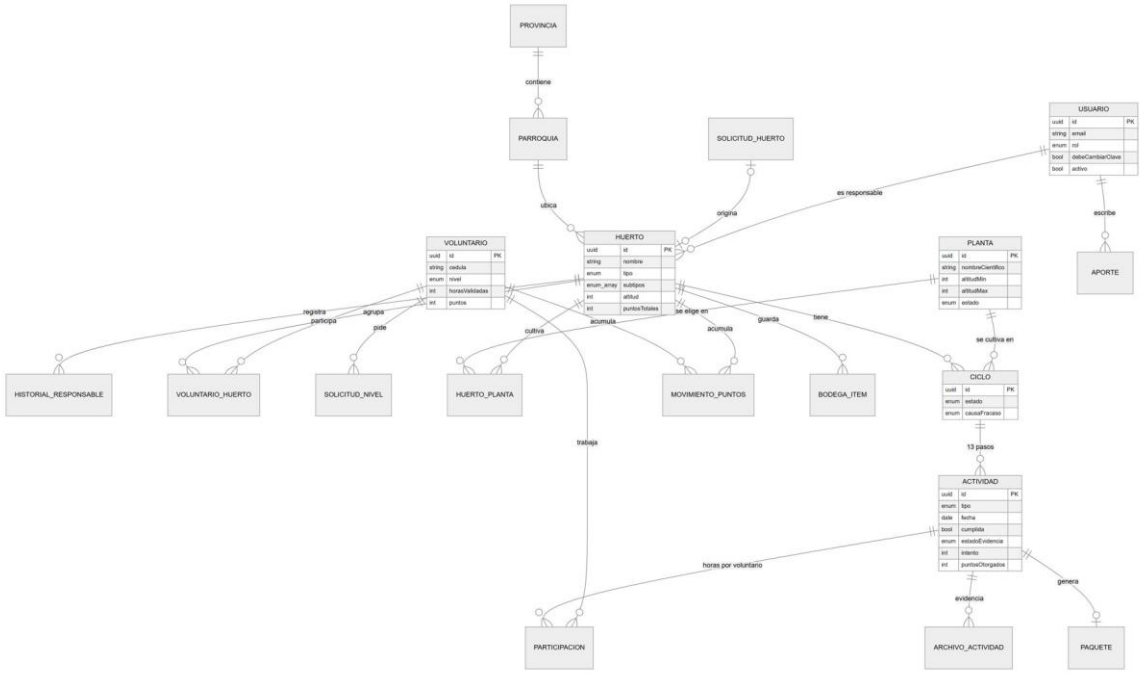


Figura 3.2. Modelo de datos

El flujo principal tiene dos partes. El responsable registra un paso del ciclo: el servidor comprueba el orden de los pasos y el límite de horas, y valida los datos con las reglas compartidas; si solo hay advertencias, el responsable las confirma y guarda. La evidencia queda pendiente. Después, el técnico del MSP la revisa; al aprobarla, una transacción acredita los puntos a cada voluntario y actualiza niveles y ranking.

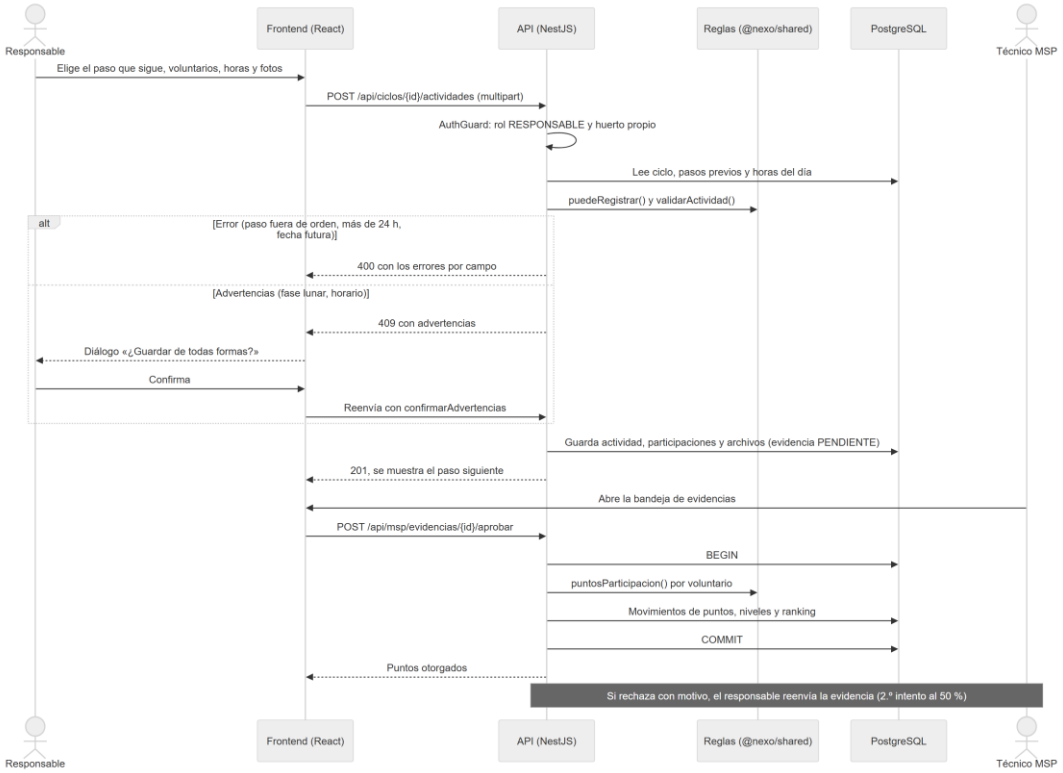


Figura 3.3. Secuencia del registro de una actividad y la validación de su evidencia

El despliegue se define en docker-compose.yml con cuatro servicios en una red interna: nexo-db, nexo-backend, nexo-frontend y nexo-mail. La base de datos y las evidencias persisten en volúmenes. Al iniciar, el backend aplica las migraciones y carga los datos de demostración. Todo el entorno se levanta con el comando docker compose up --build.

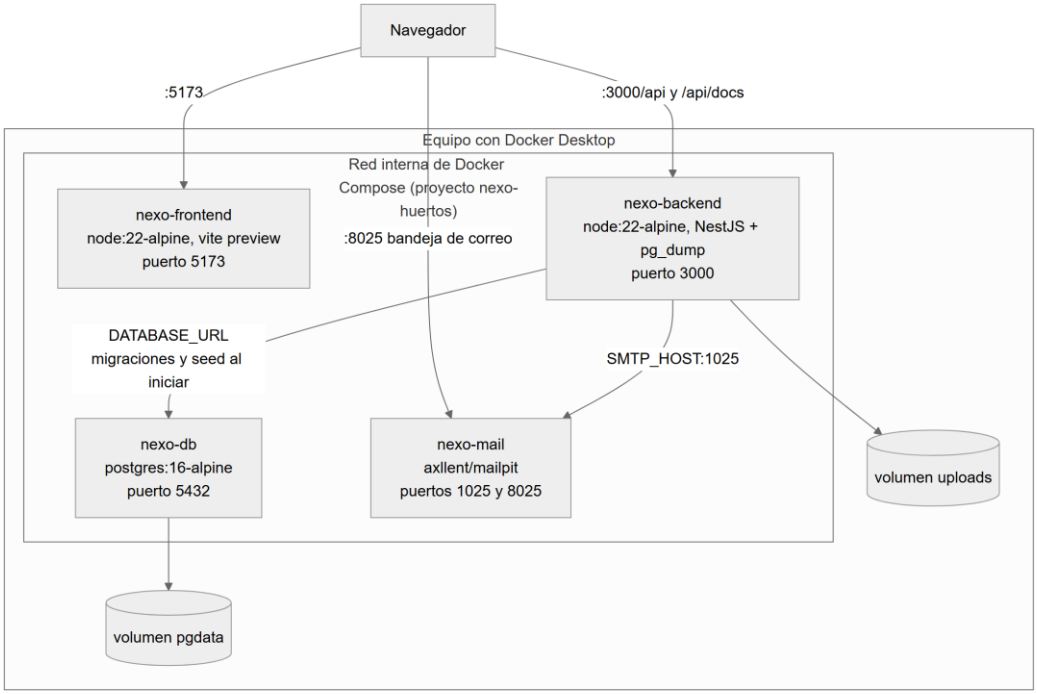


Figura 3.4. Diagrama de despliegue

Las decisiones principales fueron: separar frontend y backend mediante una API REST; usar TypeScript y reglas compartidas para evitar inconsistencias entre capas; calcular la fase lunar con un algoritmo propio, sin depender de internet; advertir sin bloquear, de modo que un error de datos impide guardar y una advertencia se confirma; y no mostrar datos personales en la parte pública, que redondea la ubicación de cada huerto a cerca de 1 km. Los permisos se verifican en el servidor con un guard que revisa el rol, que la cuenta siga activa y que se haya cambiado la contraseña temporal.

Tabla 3.1. Roles y permisos

Funcionalidad	Público	Responsable	Técnico MSP	Admin.
Ver mapa, fichas, ranking, estadísticas, catálogo, saberes y foro	✓	✓	✓	✓
Solicitar un huerto	✓			
Registrar voluntarios, plantas, actividades con evidencia, bodega y producción		✓		
Publicar en el foro		✓	✓	
Aprobar solicitudes, evidencias, plantas y niveles			✓	
Administrar el catálogo, el registro de huertos y la moderación			✓	

Funcionalidad	Público	Responsable	Técnico MSP	Admin.
Respaldo, estado del sistema y cuentas de técnicos				✓

3.2 Diseño del producto

El diseño piensa en quien usa la aplicación en el huerto. Es móvil primero, con botones de 44 a 48 píxeles, una sola columna de contenido y contraste verificado con pruebas automáticas de accesibilidad. La parte pública tiene una cabecera con Mapa, Ranking, Estadísticas, Plantas, Saberes y Foro, y el botón «Solicitar huerto». Al ingresar, cada rol ve solo sus destinos: en móvil, una barra inferior con los cuatro más usados y un menú con el resto; en escritorio, un menú lateral. Sin sesión, cualquier ruta del panel lleva al inicio de sesión, y una cuenta con contraseña temporal solo puede ir a cambiarla.

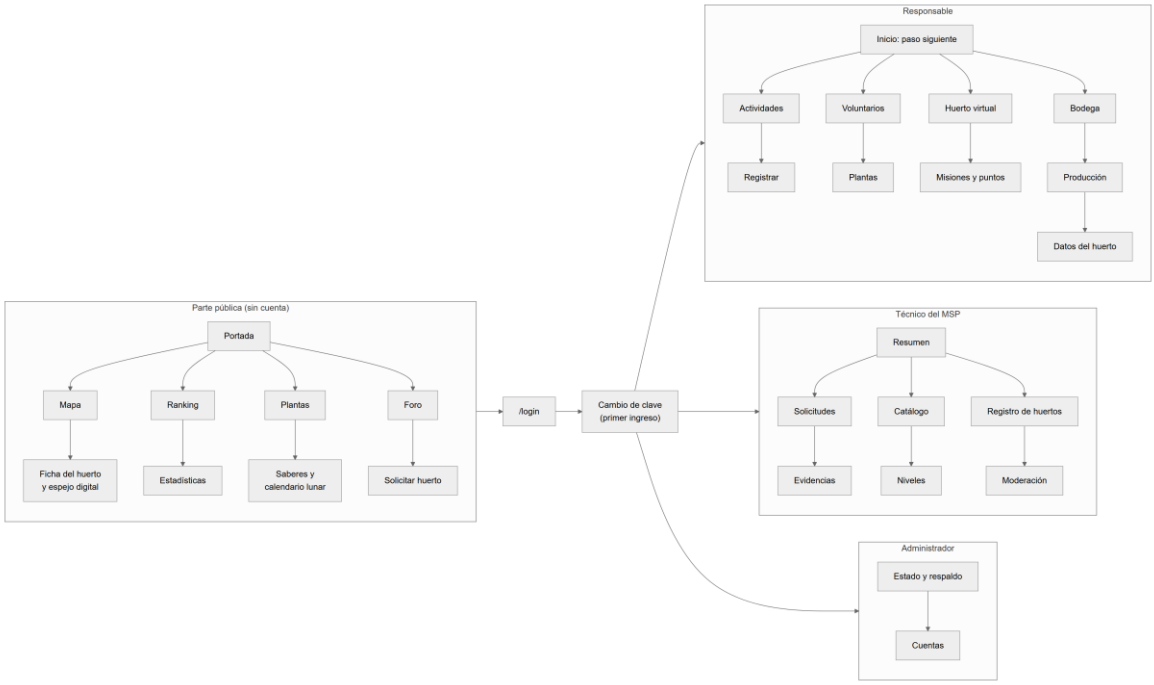


Figura 3.5. Mapa de navegación por rol

El sistema de diseño está documentado en apps/frontend/DESIGN.md. Usa fondos de tierra clara, tarjetas blancas y un único verde de acción, reservado para lo que se puede presionar. Tiene una sola familia tipográfica, Nunito, e iconos de línea de Lucide acompañados de texto, por eso el color nunca es la única señal. Los estados (pendiente, aprobada, rechazada) se muestran con texto e icono.

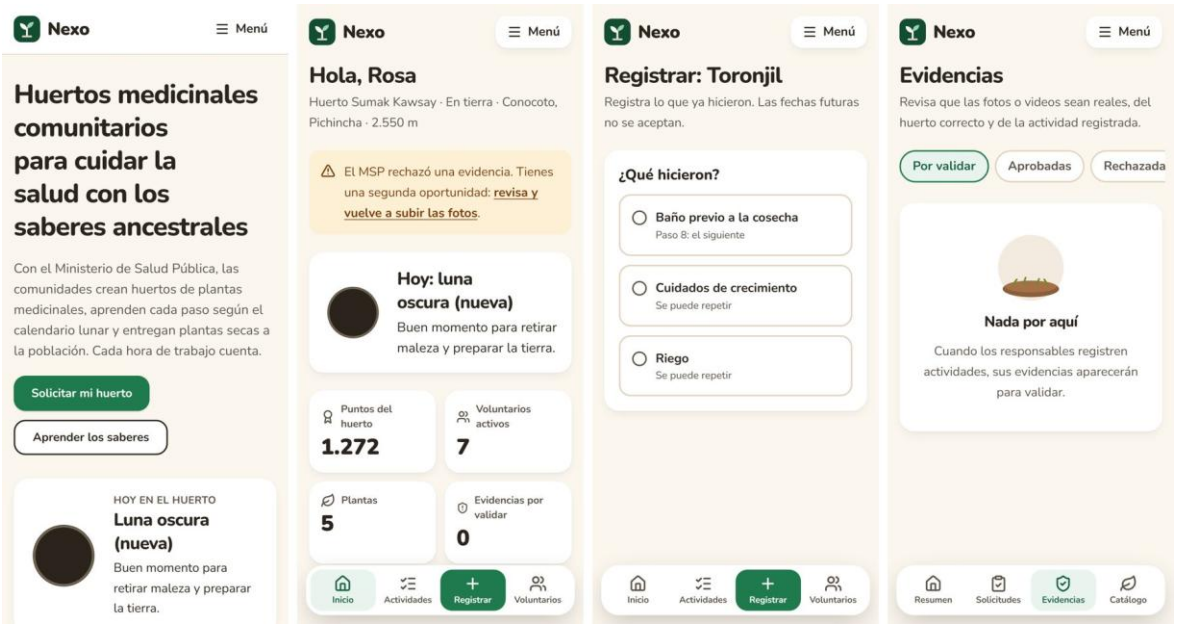


Figura 3.6. Pantallas principales en el celular: portada, inicio del responsable, registro de un paso y evidencias del MSP

El huerto virtual, o espejo digital, muestra una tarjeta por planta con un dibujo de su etapa, que depende del paso más avanzado del ciclo: sin iniciar, tierra preparada, sembrada, en cuidado, cosechada, en secado y empaquetada. Ahora cubre las siete etapas y también se ve en la ficha pública de cada huerto. El porcentaje del huerto es el promedio de avance de sus plantas.

4. Herramientas y tecnologías de desarrollo

El producto se desarrolla como una aplicación web con frontend y backend separados, escrita íntegramente en TypeScript, organizada en un monorepo con pnpm y ejecutada mediante Docker Compose. La Tabla 4.1 resume las tecnologías de desarrollo y la Tabla 4.2 las herramientas de gestión, colaboración, pruebas y diseño.

4.1 Tecnologías de desarrollo

Tabla 4.1. Tecnologías de desarrollo

Categoría	Tecnología	Uso en el proyecto	Estado
Lenguaje	TypeScript [2]	Lenguaje único en backend, frontend y paquete de reglas compartidas, con modo estricto.	En uso
Backend	NestJS 11 [3]	API REST por dominio: acceso, público, responsable, MSP, administración y foro.	En uso
Frontend	React 19 [4]	Interfaz de página única para el público, responsables, técnicos y administradores.	En uso
Estilos	Tailwind CSS 4 [5]	Sistema de diseño con tokens y componentes reutilizables.	En uso

Categoría	Tecnología	Uso en el proyecto	Estado
Base de datos	PostgreSQL 16 [6]	Huertos, voluntarios, catálogo, ciclos, actividades, puntos, foro y bodega.	En uso
ORM	Prisma 6 [7]	Esquema, migraciones, datos de demostración y cliente tipado.	En uso
Gestor de paquetes	pnpm con workspaces [8]	Monorepo con apps/backend, apps/frontend y packages/shared.	En uso
Contenedores	Docker y Docker Compose [9]	Entorno reproducible con cuatro servicios.	En uso
Compilación y rutas	Vite 7 y React Router 7	Empaquetado y navegación. Para la API se usa fetch, propio del navegador.	En uso
Autenticación y roles	JWT, bcrypt y guards de NestJS	Inicio de sesión, contraseña temporal y permisos por rol.	En uso
Documentación de la API	Swagger (@nestjs/swagger)	Documentación automática en /api/docs.	En uso
Validación de datos	class-validator	Validación de los datos de entrada, incluida la cédula ecuatoriana.	En uso
Mapas	Leaflet y OpenStreetMap [11]	Ubicar el huerto al solicitarlo y mapa público de huertos.	En uso
Correo	Nodemailer y Mailpit [12]	Envío de credenciales; Mailpit los recibe en la demo.	En uso
Archivos	Multer	Subida de fotos (10 MB) y videos (50 MB) de evidencia.	En uso
Fases lunares	Algoritmo propio en @nexo/shared	Fase lunar del día y calendario del mes, sin internet.	En uso
Calidad de código	ESLint (con jsx-a11y) y Prettier	Reglas de estilo y de accesibilidad en todo el monorepo.	En uso

4.2 Herramientas de gestión, colaboración, pruebas y diseño

Tabla 4.2. Herramientas de gestión, colaboración, pruebas y diseño

Categoría	Herramienta	Uso	Estado
Control de versiones	Git y GitHub: rama main y ramas feat/, fix/ y docs/ con pull requests	Trabajo en equipo y entrega del código	En uso
Integración continua	GitHub Actions	Tipos, pruebas de dominio, build, API y flujo e2e en cada cambio	En uso
Gestión de tareas	Bitácora (docs/bitacora.md) y pull requests	Alcance de cada versión, pendientes y avances	En uso
Diseño de interfaces	Figma	Variables, componentes y pantallas móviles de la primera versión	En uso

Categoría	Herramienta	Uso	Estado
Diagramas	Mermaid	Arquitectura, modelo de datos, flujos y navegación	En uso
Pruebas unitarias	Vitest	Reglas de dominio del paquete compartido	En uso
Pruebas de API	node:test con fetch y Swagger UI	Endpoints de punta a punta, incluido el correo en Mailpit	En uso
Pruebas de interfaz	Playwright [13] y axe-core [14]	Flujos por rol en móvil y escritorio con WCAG 2.1 AA	En uso
Auditoría web	Lighthouse [15]	Rendimiento, accesibilidad, buenas prácticas y SEO	En uso
Documentación	Plantillas de Word de la asignatura y Markdown en docs/	Informe, historias de usuario y casos de prueba	En uso
IA generativa	Claude (Anthropic) [10]	Apoyo en planificación, desarrollo, pruebas y revisión	En uso

4.3 Organización del repositorio

El proyecto se organiza como un monorepo para que todo el equipo trabaje con la misma estructura y versión de las herramientas. Cada integrante levanta el proyecto completo con el comando docker compose up --build.

```
nexo-huertos-medicinales/
├── README.md, CHANGELOG.md, docker-compose.yml, .env.example
├── apps/
│   ├── backend/     (NestJS, Prisma, migraciones y datos demo)
│   └── frontend/    (React + Tailwind, DESIGN.md)
├── packages/
│   └── shared/      (reglas de dominio y enumeraciones, con pruebas)
├── e2e/             (Playwright: flujos, accesibilidad y capturas)
└── docs/
    ├── historias-de-usuario/   (.md y .docx)
    ├── arquitectura-y-diseno/  (diagramas)
    ├── casos-de-prueba/        (.md y .docx)
    ├── informe/  y  capturas/
    ├── reunion-msp-2026-11-06.md
    └── bitacora.md, como-ejecutar.md
```

4.4 Justificación de la selección

TypeScript en ambos extremos permite compartir tipos y reglas (por ejemplo, los 13 pasos del cultivo, la fórmula de puntos y la validación de la cédula) y detectar en compilación los cambios que rompen el contrato entre frontend y backend. NestJS aporta una estructura modular que coincide con los roles del proyecto y facilita los permisos. Prisma y PostgreSQL ofrecen un modelo relacional con migraciones versionadas, adecuado para huertos, ciclos, actividades y puntuaciones. Docker Compose garantiza que todo el equipo, el tester y el evaluador ejecuten el mismo entorno, incluido el servidor de correo de prueba, y pnpm con workspaces reduce el tiempo de instalación y mantiene un solo archivo de dependencias bloqueadas.

5. Código fuente

El código está publicado en https://github.com/wesdell/nexo-huertos-medicinales. La primera entrega es la release v1.0.0 (6 de noviembre de 2026) y la versión completa es la v2.0.0 (10 de noviembre de 2026). Está escrito íntegramente en TypeScript y organizado en el monorepo descrito en la sección 4.3. GitHub Actions ejecuta en cada cambio la verificación de tipos, las pruebas de dominio, el build, las pruebas de API y el flujo de interfaz con accesibilidad sobre Docker Compose.

5.1 Estructura de carpetas

Tabla 5.1. Contenido del repositorio

Ruta	Contenido
apps/backend/src	API NestJS, un archivo por dominio: auth.ts (sesión, roles y contraseña temporal), publico.ts, huertos.ts (responsable), msp.ts (técnico), admin.ts, foro.ts, gamificacion.ts (puntos, niveles, ranking y misiones), correo.ts, archivos.ts y dto.ts
apps/backend/prisma	schema.prisma, migraciones, seed.js y catálogo inicial de 28 plantas medicinales
apps/frontend/src	SPA React: main.tsx (rutas por rol), Layout.tsx, sesion.tsx, api.ts, ui.tsx, Mapa.tsx y pages/ (público, responsable, MSP y administrador)
packages/shared/src	Reglas de dominio: pasos.ts, actividades.ts, gamificacion.ts, validaciones.ts, catalogos.ts, lunar.ts, huerto-virtual.ts y enums.ts
e2e y apps/backend/test	Pruebas de interfaz con accesibilidad (Playwright y axe), capturas y pruebas de API
docs	Historias de usuario, arquitectura y diseño, casos de prueba, informe, reunión con el MSP, bitácora y guía de ejecución

5.2 Descripción del código

El backend valida los datos de entrada con class-validator y devuelve los errores con el mismo formato que @nexo/shared (campo y mensaje). Las contraseñas se guardan con bcrypt y la sesión usa tokens JWT; el inicio de sesión responde con un mensaje genérico para no revelar qué dato falló. Aprobar una solicitud crea el huerto, la cuenta del responsable y envía el correo en una sola transacción: si el correo falla, no se aprueba nada. Aprobar una evidencia acredita los puntos, actualiza niveles y ranking también en una transacción. Las evidencias se guardan con un nombre aleatorio y se valida su tipo y tamaño; si el registro falla, los archivos subidos se borran.

El frontend usa React 19, React Router 7, Tailwind CSS 4 y Vite. Protege las rutas según el rol y reutiliza las reglas de @nexo/shared para mostrar el paso siguiente y las advertencias antes de guardar. Los mapas usan Leaflet con mosaicos de OpenStreetMap.

El paquete compartido contiene funciones puras y probadas: los 13 pasos del cultivo y su orden, la validación de cada paso según los lineamientos del MSP, la fase lunar, los puntos por hora, los niveles, los talentos, el ranking, las misiones, la validación de la cédula ecuatoriana y el huerto virtual. Una prueba comprueba que las enumeraciones coinciden con las de la base de datos.

5.3 Instrucciones de ejecución

El único requisito es tener Docker Desktop abierto. Desde una terminal:

```
git clone https://github.com/wesdell/nexo-huertos-medicinales.git
```

```
cd nexo-huertos-medicinales
cp .env.example .env
docker compose up --build
```

Si en ese equipo ya se ejecutó la versión 1, antes hay que borrar sus datos con docker compose down -v, porque la versión 2.0.0 cambia el modelo de datos. La aplicación queda disponible en http://localhost:5173, la documentación de la API en http://localhost:3000/api/docs y la bandeja de correos de prueba en http://localhost:8025. Para las pruebas se usan estos comandos (los de API y de interfaz requieren la aplicación en ejecución):

```
pnpm --filter @nexo/shared test    # reglas de dominio
pnpm --filter backend test:e2e     # API de punta a punta
pnpm e2e                           # interfaz y accesibilidad
node e2e/capturas-flujo.mjs        # recorrido del flujo básico con capturas
```

Las cuentas de prueba y la verificación del sistema se detallan en la sección 7; la solución de problemas, en docs/como-ejecutar.md.

6. Casos de prueba y resultados

Responsable de pruebas: Josselyn Pozo

Entorno de evaluación: local, con Docker Compose (Node.js 22, React, NestJS, PostgreSQL 16 y Mailpit), en los navegadores Chromium para móvil (412 px) y escritorio (1440 px).

Fecha de ejecución: 9 de noviembre de 2026.

6.1 Resumen de ejecución

Se ejecutaron las pruebas funcionales, de interfaz, de accesibilidad y de calidad web sobre la versión 2.0.0. Se verificó el flujo completo: la solicitud pública del huerto, su aprobación con el correo de credenciales, el cambio de contraseña, el registro de voluntarios y plantas, el registro de los pasos con evidencia, la validación del MSP con la acreditación de puntos, los rankings, la parte pública y la administración. Los 20 casos documentados con la plantilla están en docs/casos-de-prueba/casos-de-prueba.docx.

Tabla 6.1. Resumen de los casos de prueba

Total de casos	Aprobados	Fallidos	Bloqueados	Cobertura
20	20	0	0	100 % de las HU nuevas (HU-037 a HU-052)

Tabla 6.2. Pruebas automáticas

Nivel	Herramienta	Qué verifica	Resultado
Unitarias	Vitest	Reglas de dominio: pasos, validaciones, puntos, niveles, ranking, cédula y fase lunar	66 de 66
API de punta a punta	node:test	Solicitud, aprobación con correo real en Mailpit, roles, voluntarios, altitud, pasos, evidencias, ranking, foro y respaldo	13 de 13

Nivel	Herramienta	Qué verifica	Resultado
Interfaz y accesibilidad	Playwright y axe-core	Cuatro flujos por rol en móvil y escritorio, sin violaciones serias de WCAG 2.1 AA	8 de 8
Estáticas	TypeScript y ESLint	Tipos estrictos y reglas de accesibilidad jsx-a11y	Sin errores
Auditoría web	Lighthouse 12	13 pantallas en móvil y escritorio	96 a 100
Recorrido completo	Playwright	Flujo básico de punta a punta con 35 capturas	Sin errores

6.2 Detalle de los casos de prueba

Tabla 6.3. Casos de prueba ejecutados

ID	Caso	HU	Nivel	Estado
P-001	Solicitud de huerto con datos inválidos	HU-037	API y e2e	Aprobado
P-002	Solicitud válida y duplicada	HU-037	API	Aprobado
P-003	Aprobación con contraseña temporal por correo	HU-038, 039	API	Aprobado
P-004	Rechazo de solicitud con motivo	HU-038	API	Aprobado
P-005	Permisos por rol	HU-002, 051	API	Aprobado
P-006	Voluntario sin cuenta y nivel por experiencia	HU-040, 041	API	Aprobado
P-007	Planta de otra altitud	HU-042	API	Aprobado
P-008	Pasos en orden, evidencia obligatoria y 24 horas	HU-044	API	Aprobado
P-009	Segunda oportunidad de la evidencia	HU-045	API	Aprobado
P-010	Paso no cumplido y cosecha perdida	HU-046	API	Aprobado
P-011	Ranking normalizado por tipo	HU-048	Unitaria y API	Aprobado
P-012	Niveles automáticos y talentos	HU-047	Unitaria	Aprobado
P-013	Validación de cada paso según los lineamientos	HU-044, 052	Unitaria	Aprobado
P-014	Información pública sin datos personales	HU-049	API y e2e	Aprobado
P-015	Foro con reglas y moderación	HU-008, 009	API	Aprobado
P-016	Respaldo y estado del sistema	HU-051	API	Aprobado
P-017	Carga masiva del catálogo	HU-043	API	Aprobado
P-018	Accesibilidad y diseño adaptable	Todas	e2e con axe	Aprobado
P-019	Rendimiento y calidad web con Lighthouse	Todas	Auditoría	Aprobado
P-020	Flujo básico completo en la interfaz	HU-037 a 051	e2e	Aprobado

A continuación se detallan los casos que muestran las reglas acordadas con el MSP. Las pantallas de cada uno aparecen en el recorrido de la sección 7.4.

Caso de prueba P-003: Aprobación con contraseña temporal por correo

- **Descripción:** Verificar que, al aprobar una solicitud, se cree la cuenta del responsable, llegue un correo con su contraseña temporal y el sistema obligue a cambiarla.
- **Resultado esperado:** Correo en la bandeja de Mailpit; con la contraseña temporal el panel responde 403 hasta que se cambia, y 200 después.
- **Resultado obtenido:** Exitoso. El correo llegó con el usuario y la contraseña temporal (Figura 7.4) y el primer ingreso llevó a «Crea tu contraseña» (Figura 7.5).
- **Estado:** APROBADO

Caso de prueba P-007: Planta de otra altitud

- **Descripción:** Comprobar que el catálogo se filtra por la altitud del huerto y que elegir una planta de otra altitud pide confirmación.
- **Resultado esperado:** El servidor responde 409 con la advertencia de altitud; si se confirma, la planta se agrega marcada como fuera de altitud.
- **Resultado obtenido:** Exitoso. La guayusa (200 a 2000 m) en un huerto a 2850 m mostró «¿Estás seguro?» (Figura 7.6).
- **Estado:** APROBADO

Caso de prueba P-008: Pasos en orden, evidencia obligatoria y 24 horas

- **Descripción:** Validar que un paso no se registre antes del anterior, que exija fotos o videos y que nadie supere 24 horas en un día.
- **Resultado esperado:** Paso fuera de orden: 409. Sin evidencia: 400. Más de 24 horas: 400. Datos correctos: 201.
- **Resultado obtenido:** Exitoso. Los cuatro resultados coinciden con lo esperado.
- **Estado:** APROBADO

Caso de prueba P-009: Segunda oportunidad de la evidencia

- **Descripción:** Verificar que una evidencia rechazada se pueda reenviar una vez y que otorgue la mitad de los puntos.
- **Resultado esperado:** Primer intento rechazado con motivo; el segundo, aprobado, otorga 2 h × 10 puntos × 50 % × 1,1 (bono lunar) = 11 puntos.
- **Resultado obtenido:** Exitoso. 11 puntos y 2 horas validadas para la voluntaria.
- **Estado:** APROBADO

Caso de prueba P-011: Ranking normalizado por tipo

- **Descripción:** Comprobar que el índice del ranking sigue la fórmula propuesta y que solo compara huertos del mismo tipo.
- **Resultado esperado:** Índice = puntos de 12 meses ÷ √(voluntarios activos + 1); un tipo inválido responde 400.
- **Resultado obtenido:** Exitoso. Los índices publicados coinciden con la fórmula (Figura 7.12).
- **Estado:** APROBADO

6.3 Rendimiento y accesibilidad con Lighthouse

Lighthouse [15] mide de 0 a 100 el rendimiento, la accesibilidad, las buenas prácticas y el posicionamiento en buscadores (SEO) de una página. Se auditaron 13 pantallas, nueve públicas y cuatro con sesión, en modo móvil (celular simulado con red lenta) y en modo escritorio, con la aplicación compilada para producción en Docker. La Tabla 6.4 muestra los resultados.

Tabla 6.4. Resultados de Lighthouse (móvil / escritorio)

Pantalla	Rendimiento	Accesibilidad	Buenas prácticas	SEO
Portada	100 / 100	100 / 100	100 / 100	100 / 100
Mapa de huertos	100 / 100	100 / 100	96 / 100	100 / 100
Ranking	100 / 100	100 / 100	100 / 100	100 / 100
Estadísticas	100 / 100	100 / 100	100 / 100	100 / 100
Catálogo de plantas	100 / 100	100 / 100	100 / 100	100 / 100
Saberes y calendario lunar	100 / 100	100 / 100	100 / 100	100 / 100
Foro	100 / 100	100 / 100	100 / 100	100 / 100
Solicitar huerto	99 / 100	100 / 100	96 / 100	100 / 100
Inicio de sesión	100 / 100	100 / 100	100 / 100	100 / 100
Responsable: inicio	100 / 100	100 / 100	100 / 100	100 / 100
Responsable: registrar	100 / 100	100 / 100	100 / 100	100 / 100
Técnico MSP: evidencias	100 / 100	100 / 100	100 / 100	100 / 100
Administrador: estado	100 / 100	100 / 100	100 / 100	100 / 100

Accesibilidad y SEO obtuvieron 100 en las 26 mediciones. Las buenas prácticas bajan a 96 solo en «Mapa» y «Solicitar huerto» en el celular, porque los mosaicos de OpenStreetMap son de 256 píxeles y se ven con menos resolución en pantallas de alta densidad; no depende del código del proyecto.

En la primera medición, el rendimiento en el celular bajó a entre 92 y 95 en cinco pantallas (portada, mapa, estadísticas, catálogo y foro). La causa fue un salto de diseño (CLS de hasta 0,87): el contenido llegaba de la API después de dibujar la página y empujaba hacia abajo el pie de página y otras

secciones. Se reservó el espacio de lo que se carga, el pie de página empieza fuera de la pantalla y en el foro las reglas se muestran antes de la lista. Al volver a medir, el rendimiento quedó entre 99 y 100.

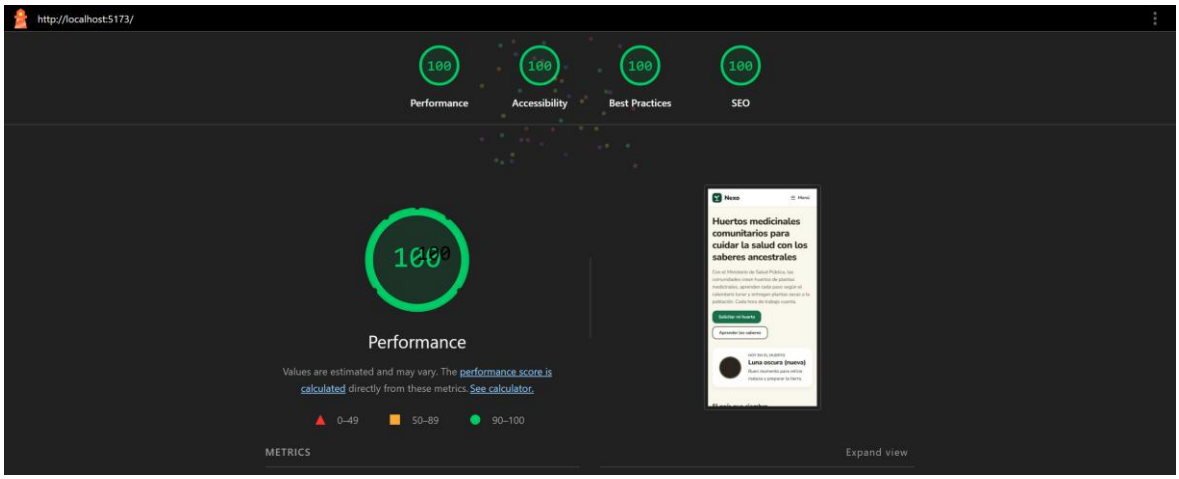


Figura 6.1. Lighthouse en la portada, modo móvil, después de la corrección

6.4 Defectos encontrados y corregidos

Tabla 6.5. Defectos encontrados en esta versión

#	Defecto	Cómo se encontró	Corrección
1	La cabecera pública se desbordaba a 1024 px de ancho	Revisión en el navegador	El menú completo aparece desde 1280 px; antes, un botón «Menú»
2	El mapa quedaba por encima de la cabecera fija al desplazarse	Revisión en el navegador	Contexto de apilamiento propio para el mapa
3	Campo de archivo oculto sin nombre accesible en «Actividades»	axe en Playwright	Nombre accesible y fuera del orden de tabulación
4	Opción de un grupo sin texto accesible	ESLint jsx-a11y	Etiqueta asociada al campo
5	Carácter invisible (BOM) escrito literal en el código del CSV	ESLint	Se escribe como secuencia de escape
6	Dos pruebas en paralelo validaban la misma evidencia	Playwright	Pruebas en serie y mensaje si otra persona ya la revisó
7	En los datos demo solo los primeros voluntarios tenían horas	Revisión de la pantalla	El seed reparte las participaciones
8	Saltos de diseño al cargar datos en el celular (CLS hasta 0,87)	Lighthouse	Espacio reservado y pie de página fuera de la pantalla inicial
9	La prueba de API fallaba a veces: generaba cédulas con tercer dígito 6 o mayor, que no son válidas	Ejecución repetida de las pruebas	El generador usa un tercer dígito de 0 a 5; tres ejecuciones seguidas en verde

6.5 Documentación de la API

La API publica su documentación interactiva en http://localhost:3000/api/docs, generada con Swagger. Lista los endpoints agrupados por dominio (acceso, público, responsable, MSP, administración y foro) y permite probarlos con un token.

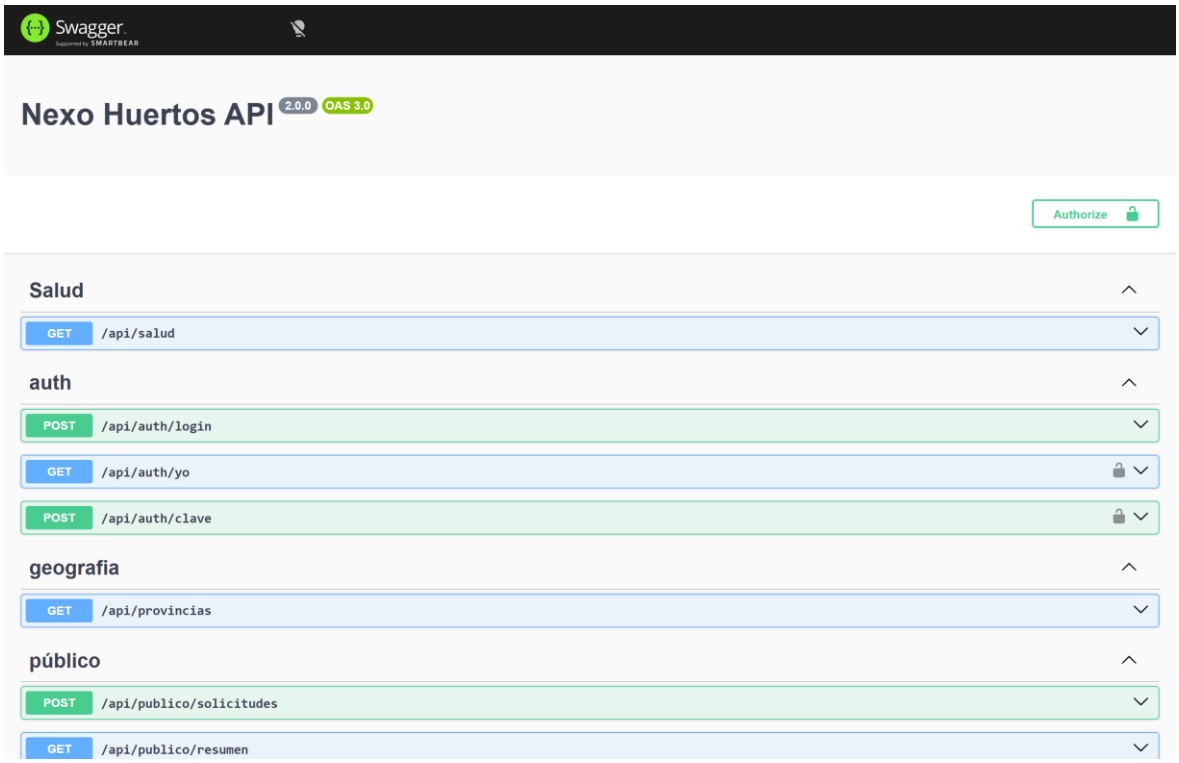


Figura 6.2. Documentación interactiva de la API en Swagger

7. Producto operativo

7.1 Acceso al sistema y URLs de verificación

Con el entorno en ejecución, la plataforma se usa desde estas direcciones locales:

- **Aplicación web:** http://localhost:5173, acceso principal a la plataforma.
- **Documentación de la API (Swagger):** http://localhost:3000/api/docs, endpoints y servicios web.
- **Bandeja de correo de prueba (Mailpit):** http://localhost:8025, donde llegan los correos con las contraseñas temporales de los responsables aprobados.

7.2 Cuentas de prueba

Los datos de demostración crean una cuenta por rol, todas con la contraseña HuertoDemo2026, ocho huertos en varias provincias, voluntarios en distintos niveles, ciclos en distintos pasos, evidencias pendientes, solicitudes pendientes y aportes del foro. Son cuentas ficticias, solo para la demo.

Tabla 7.1. Cuentas de prueba

Rol	Correo	Contraseña	Qué se puede probar
Responsable de huerto	demo@nexo.test	HuertoDemo2026	Huerto Sumak Kawsay con voluntarios, plantas, ciclos y puntos
Técnico del MSP	tecnico@nexo.test	HuertoDemo2026	Solicitudes, evidencias, catálogo y niveles pendientes de revisar
Administrador técnico	admin@nexo.test	HuertoDemo2026	Estado del sistema, respaldo y cuentas de técnicos
Público	Sin cuenta	—	Mapa, fichas, ranking, estadísticas, catálogo, saberes, foro y solicitud

7.3 Instrucciones para el evaluador

1. Abrir Docker Desktop y, en la carpeta del proyecto, ejecutar docker compose up --build. La primera vez tarda unos minutos; está listo cuando los cuatro contenedores aparecen en ejecución (Figura 7.21).

2. Si en ese equipo ya se ejecutó la versión 1, ejecutar antes docker compose down -v. El mismo comando sirve para volver a los datos demo originales después de probar.

3. Abrir http://localhost:5173. La portada es pública: se puede recorrer el mapa, el ranking y el resto sin cuenta.

4. Seguir el flujo básico de la sección 7.4: solicitar un huerto, aprobarlo con la cuenta del técnico, leer el correo en http://localhost:8025 e ingresar como el nuevo responsable.

5. Para ver un huerto con historia, ingresar con demo@nexo.test y registrar un paso; luego validarlo con tecnico@nexo.test y ver los puntos en «Misiones y puntos» y en el ranking.

7.4 Flujo básico del sistema

La Figura 7.1 resume el flujo completo, desde que una persona de la comunidad solicita su huerto hasta que su trabajo se ve en el ranking público. Las figuras siguientes muestran cada paso en la aplicación, tal como lo recorrió la prueba P-020 el 9 de noviembre de 2026.

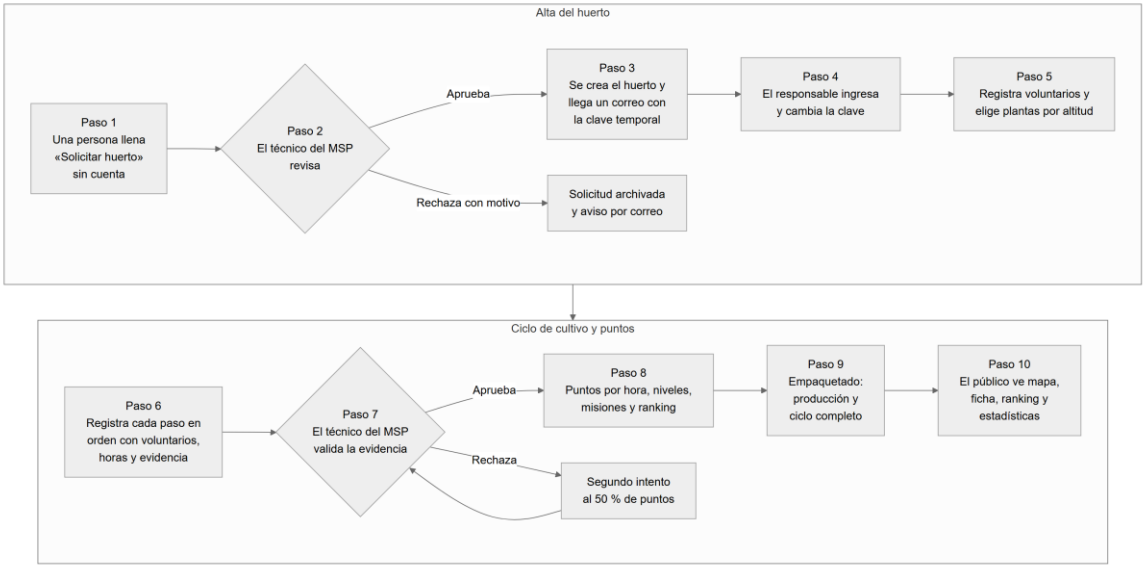


Figura 7.1. Flujo básico del sistema

Paso 1. Solicitar el huerto. Desde «Solicitar huerto», sin cuenta, la persona indica el nombre, el tipo y hasta tres subtipos, marca la ubicación en el mapa o con el GPS del celular, la altitud y sus datos: nombre, cédula, celular y correo. Debe aceptar la política de privacidad. El sistema valida la cédula y el celular antes de enviar.

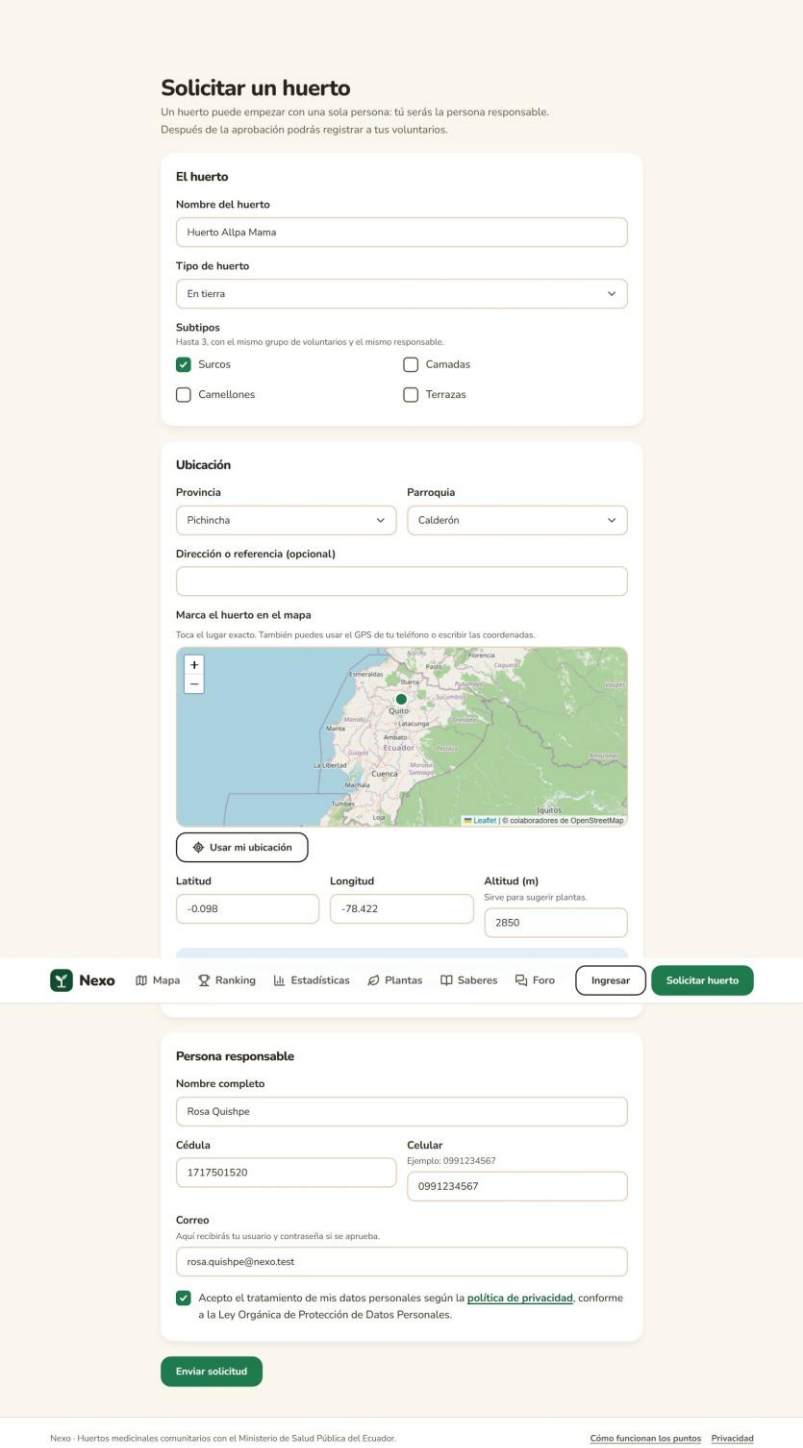


Figura 7.2. Formulario «Solicitar un huerto»

Paso 2. El técnico del MSP revisa. En «Solicitudes» el técnico ve los datos de contacto y la ubicación. Puede aprobar o rechazar con un motivo, que se envía por correo.

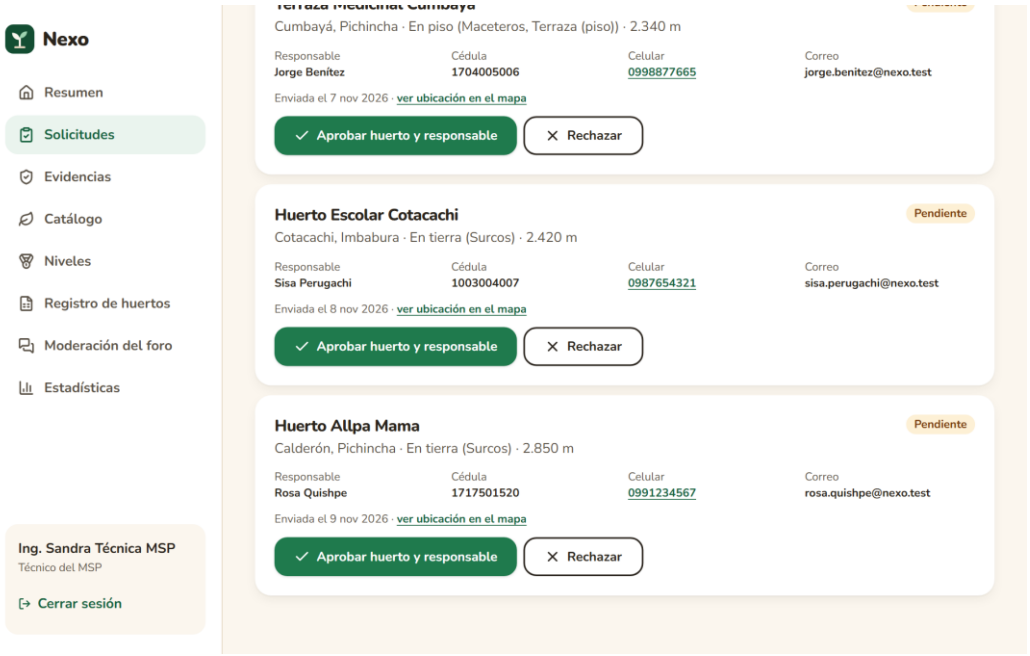


Figura 7.3. Solicitud pendiente en la bandeja del técnico del MSP

Paso 3. Llega el correo con la contraseña temporal. Al aprobar, se crea el huerto y la cuenta del responsable, y se envía un correo con el usuario y una contraseña temporal. En la demo, el correo se ve en Mailpit.

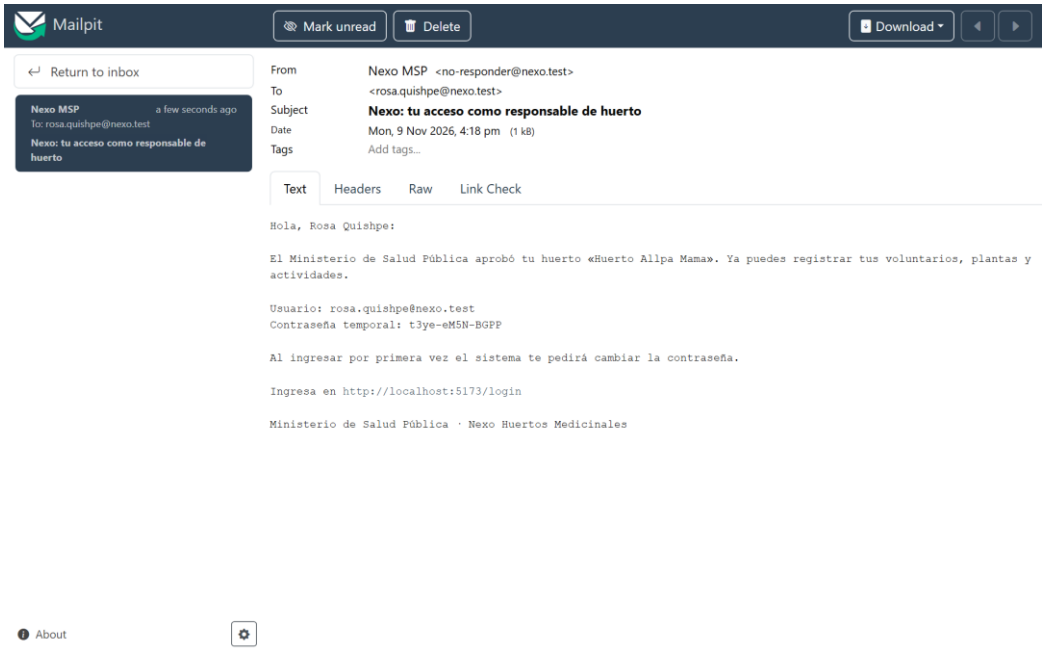


Figura 7.4. Correo con las credenciales, en la bandeja de Mailpit

Paso 4. Primer ingreso. Con la contraseña temporal, el sistema solo permite crear una contraseña nueva. Después, el responsable llega a su inicio, que le indica qué hacer primero.

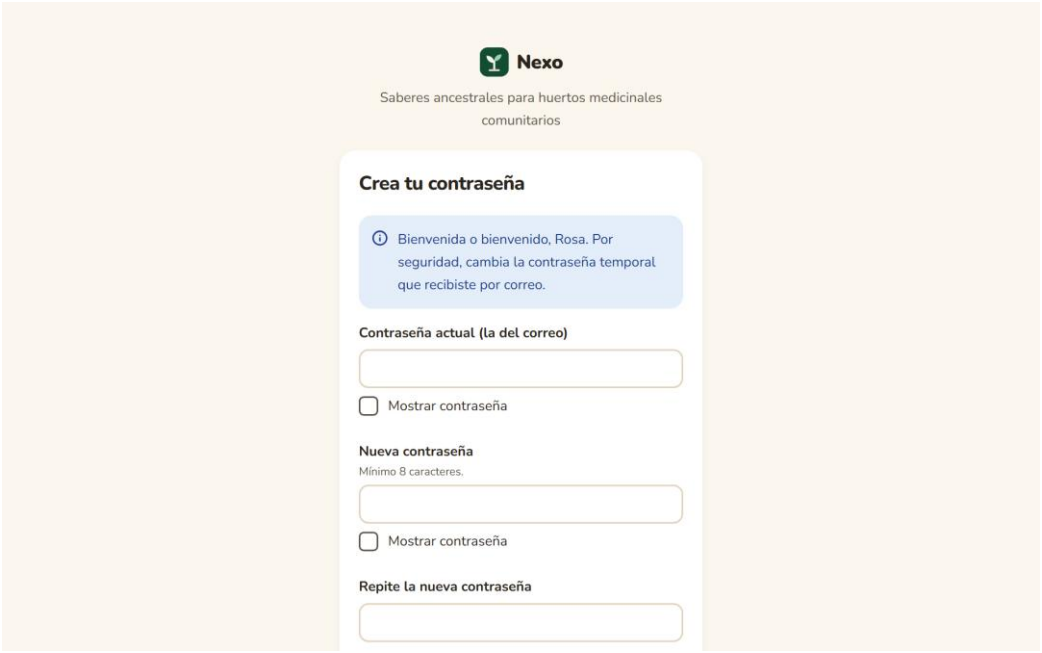


Figura 7.5. Cambio obligatorio de la contraseña temporal

Paso 5. Voluntarios y plantas. El responsable registra a sus voluntarios, que no necesitan cuenta, y elige plantas del catálogo del MSP filtradas por la altitud del huerto. Si elige una de otra altitud, el sistema pregunta antes de agregarla.

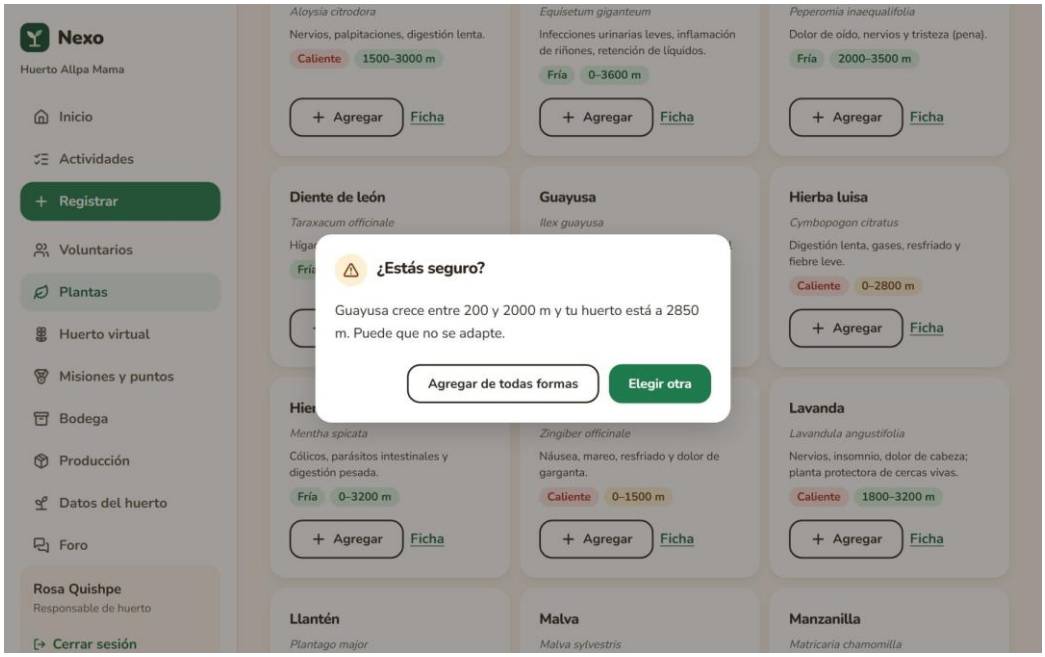


Figura 7.6. Aviso al elegir una planta de otra altitud

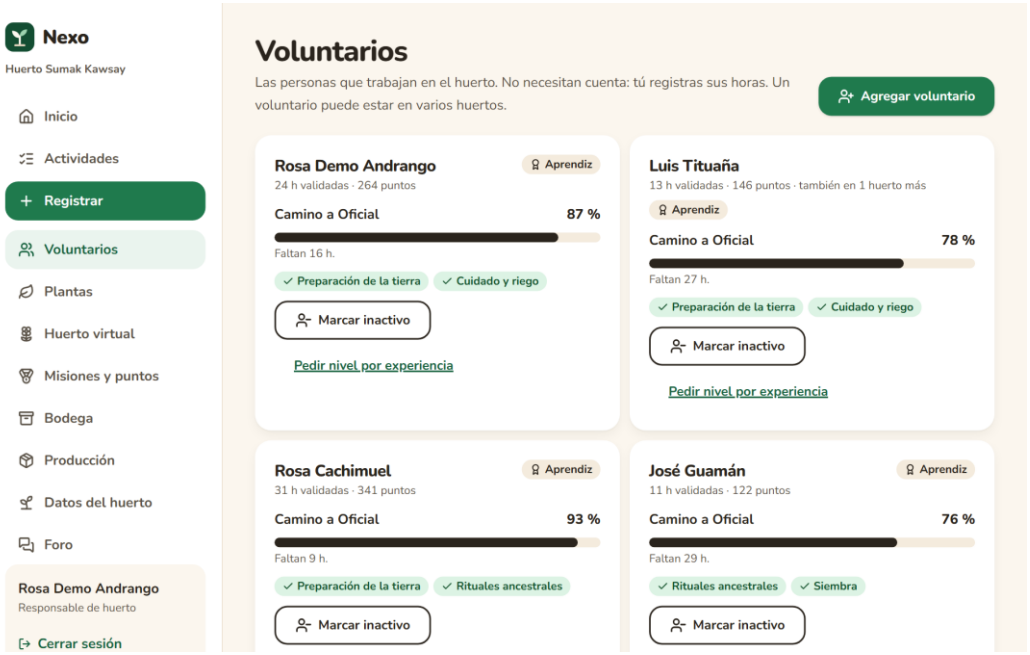


Figura 7.7. Voluntarios del huerto, con su nivel y su camino al siguiente

Paso 6. Registrar cada paso del ciclo. El sistema muestra el paso que sigue con su «cuándo» y su «cómo», tomados de los lineamientos del MSP. El responsable indica la fecha, los datos del paso,

quién participó, cuántas horas y sube fotos o videos. Si algo no sigue el calendario lunar, el sistema advierte y deja confirmar. El huerto virtual avanza con cada paso.

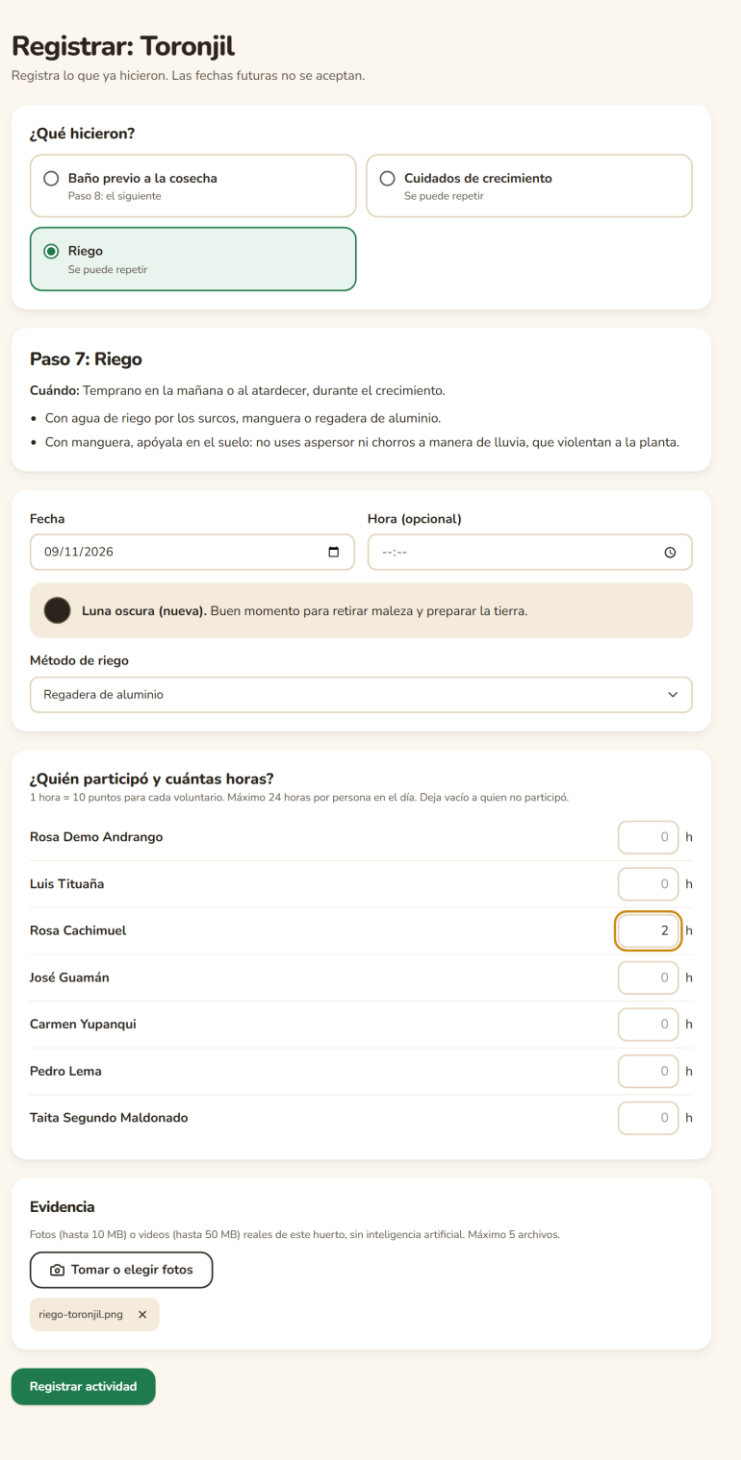


Figura 7.8. Registro de un riego con voluntarios, horas y evidencia

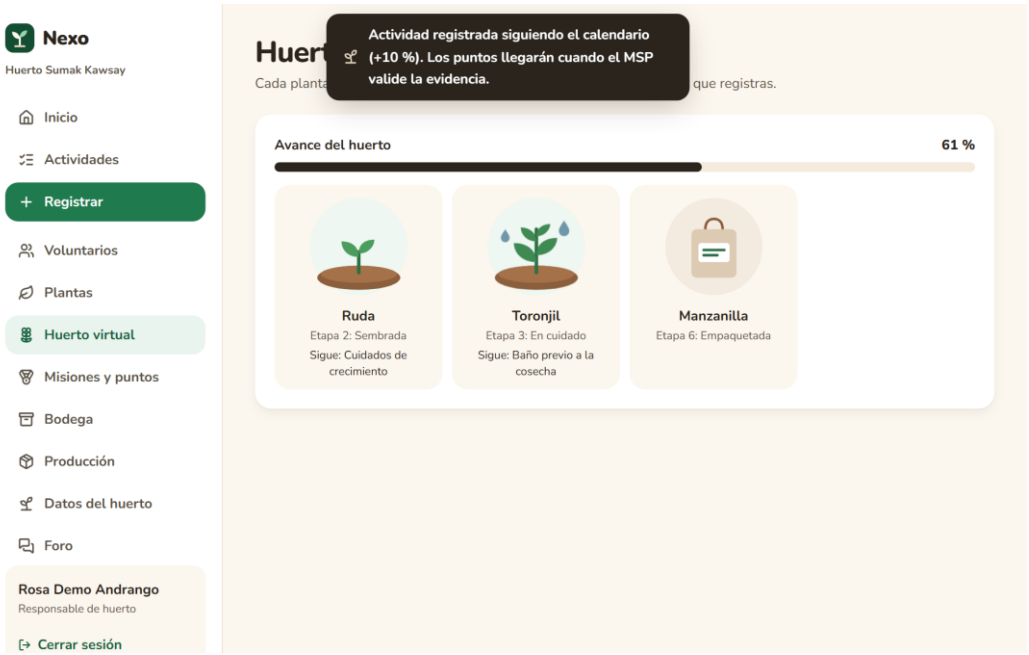


Figura 7.9. Huerto virtual después de registrar el paso

Paso 7. El técnico valida la evidencia. En «Evidencias» el técnico revisa que las fotos sean reales y del huerto correcto. Si es válida, se otorgan los puntos; si no, explica el motivo y el responsable tiene una segunda oportunidad al 50 %.

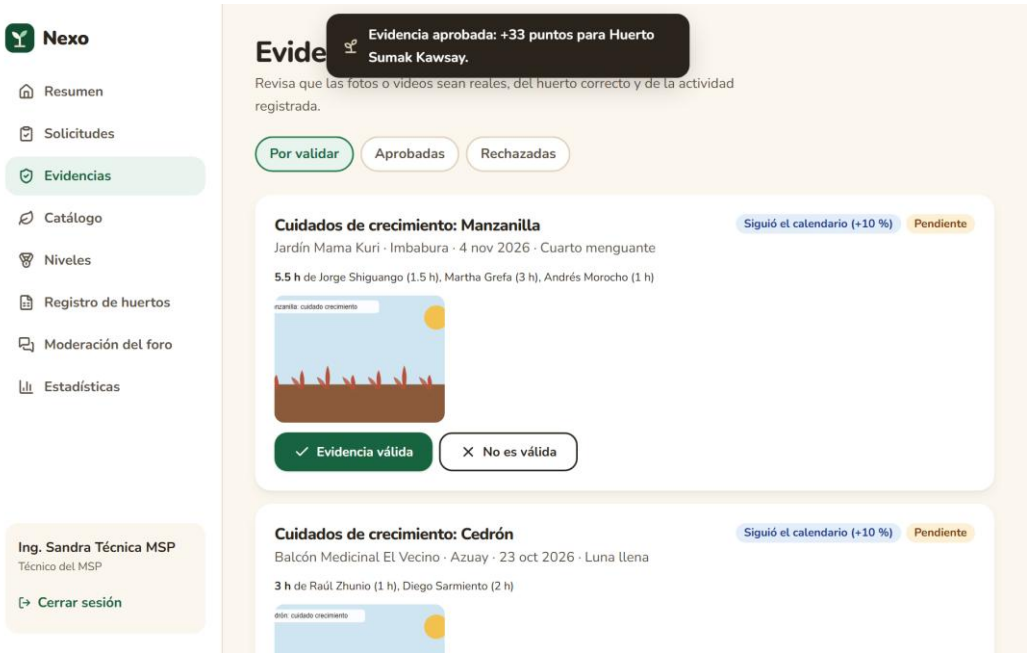


Figura 7.10. Evidencia aprobada y puntos otorgados

Paso 8. Puntos, niveles, misiones y ranking. Cada voluntario recibe 10 puntos por hora validada. El responsable ve el avance en «Misiones y puntos» y el huerto sube en el ranking de su tipo.



Figura 7.11. Misiones y puntos del huerto

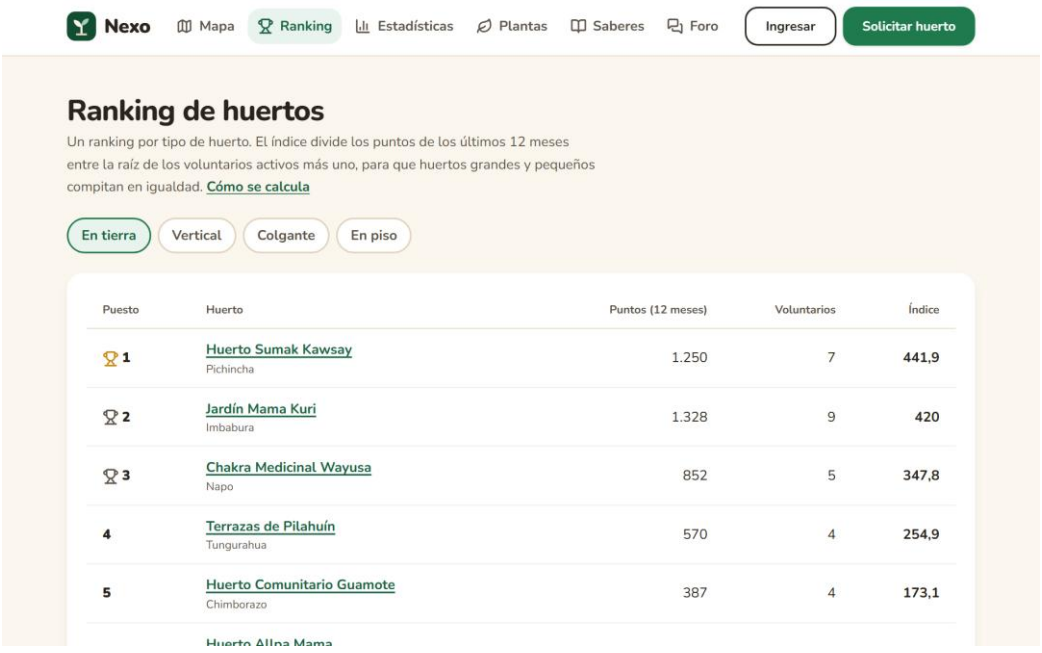


Figura 7.12. Ranking público por tipo de huerto

Pasos 9 y 10. Producción y vista pública. Al empaquetar, el huerto registra paquetes con su etiqueta y fecha de caducidad, que cuentan para el ranking de producción. Cualquier persona puede ver la ficha pública del huerto con su espejo digital, sin datos personales.

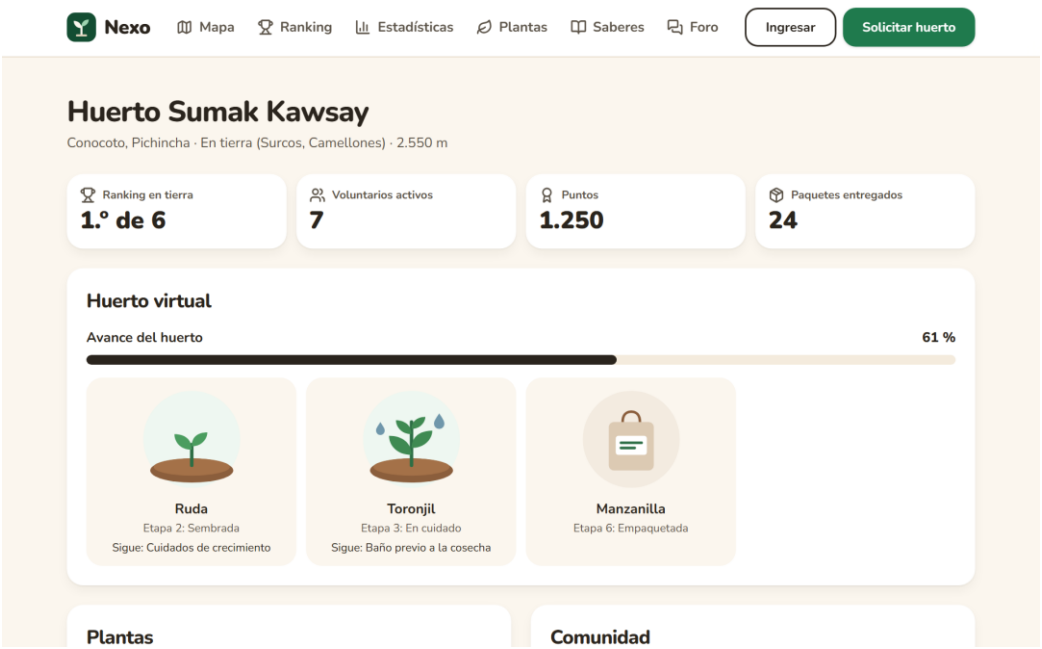


Figura 7.13. Ficha pública del huerto con su espejo digital

7.5 Funciones por rol

Cada rol ve solo lo que le corresponde. Esta sección resume qué puede hacer cada uno.

Público (sin cuenta)

- **Portada:** qué es el proyecto, la recomendación del día según la luna y las cifras del país.
- **Mapa de huertos:** ubicación aproximada de cada huerto, con filtro por tipo.
- **Ficha del huerto:** espejo digital, plantas, historia visual y puntos, sin datos personales.
- **Ranking:** por tipo de huerto y de producción del año.
- **Estadísticas:** huertos por provincia y por tipo, voluntarios por nivel y paquetes por año.
- **Catálogo de plantas y saberes:** fichas de las plantas, los 13 pasos con sus indicaciones y el calendario lunar del mes.
- **Foro:** lectura de prácticas, saberes y sentires, con sus reglas de participación.
- **Solicitar huerto:** el inicio del flujo básico.

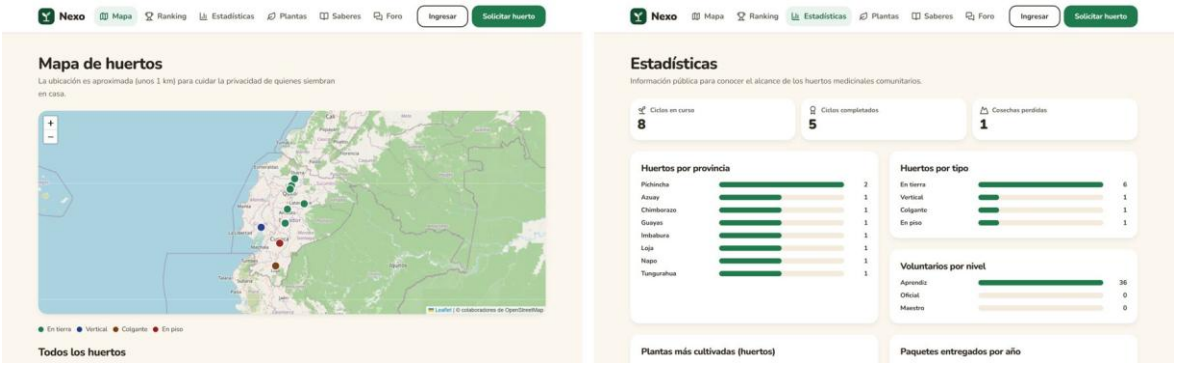


Figura 7.14. Público: mapa de huertos y estadísticas



Figura 7.15. Público: saberes con el calendario lunar y foro

Responsable del huerto

- **Inicio:** el paso que sigue en cada planta, la fase lunar y los puntos del huerto.
- **Actividades y Registrar:** ciclos por planta, registro de cada paso con evidencia, paso no cumplido y ciclo fallido con su causa.
- **Voluntarios:** agregar, marcar inactivos y pedir al MSP un nivel por experiencia.
- **Plantas:** elegir del catálogo por altitud y proponer una planta nueva.
- **Huerto virtual y Misiones y puntos:** avance de cada planta, 10 misiones con insignia, nivel y talentos de cada voluntario.
- **Bodega, Producción y Datos del huerto:** herramientas, semillas y abonos; paquetes con etiqueta; suelo, agua, abono, cerca y área.
- **Foro:** publicar y responder aportes.

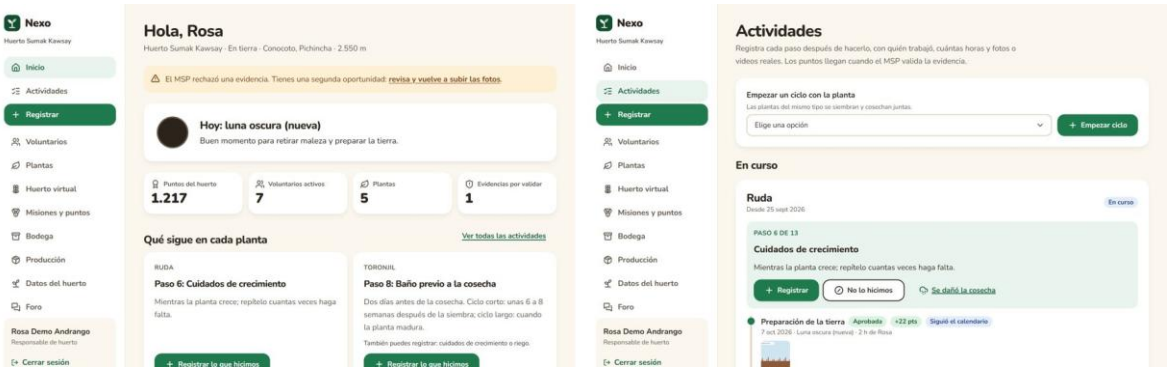


Figura 7.16. Responsable: inicio y actividades

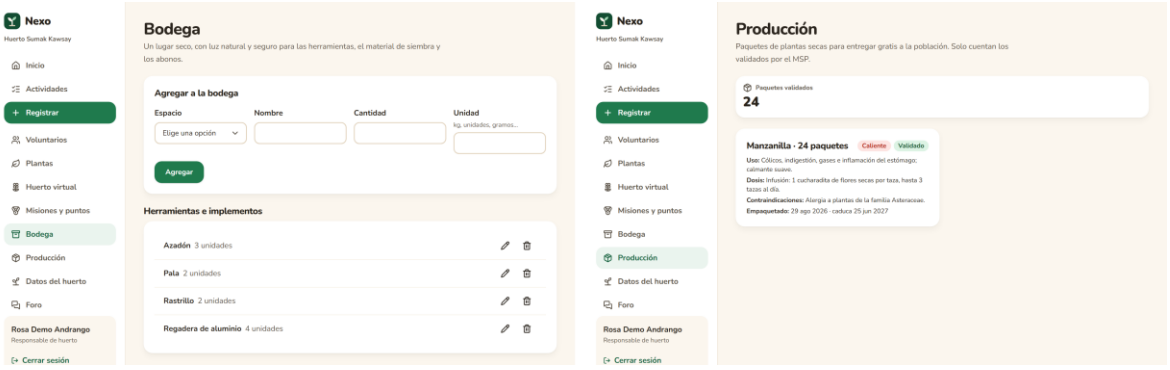


Figura 7.17. Responsable: bodega y producción

Técnico del MSP

- **Resumen:** lo que espera revisión.
- **Solicitudes:** aprobar o rechazar huertos con motivo.
- **Evidencias:** validar fotos y videos; aprobar otorga los puntos.
- **Catálogo:** crear y editar plantas, cargar varias por CSV y aprobar las propuestas de los huertos.
- **Niveles:** aprobar o rechazar los niveles por experiencia.
- **Registro de huertos:** lista centralizada exportable a CSV, cambio de responsable, restablecer contraseña y activar o desactivar huertos.
- **Moderación:** retirar o restaurar aportes reportados del foro.

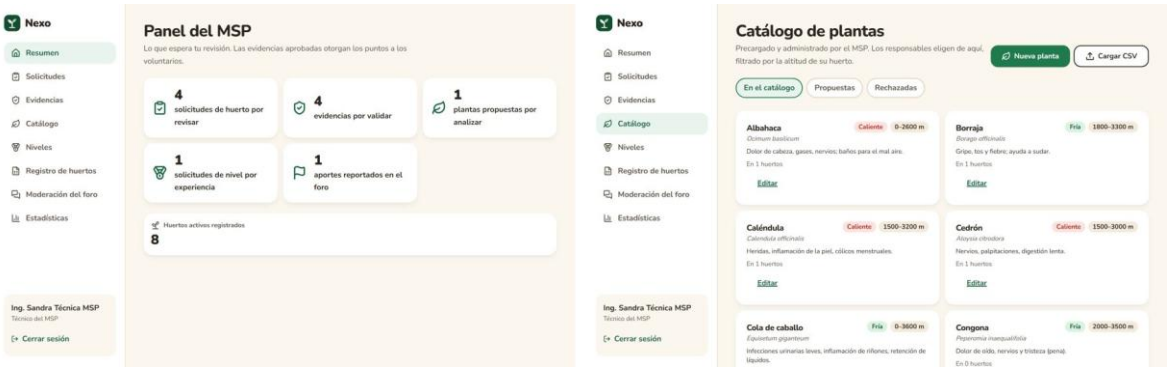


Figura 7.18. Técnico del MSP: resumen y catálogo

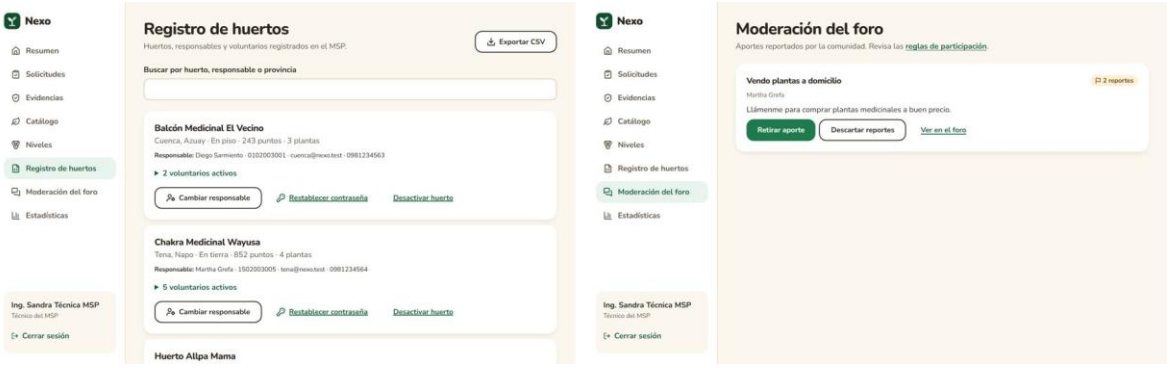


Figura 7.19. Técnico del MSP: registro de huertos y moderación del foro

Administrador técnico

- **Estado del sistema:** conexión y tamaño de la base de datos, archivos subidos y cantidad de registros por tabla.
- **Respaldo:** descarga de una copia completa de la base de datos (pg_dump).
- **Cuentas:** crear y desactivar cuentas de técnicos del MSP.

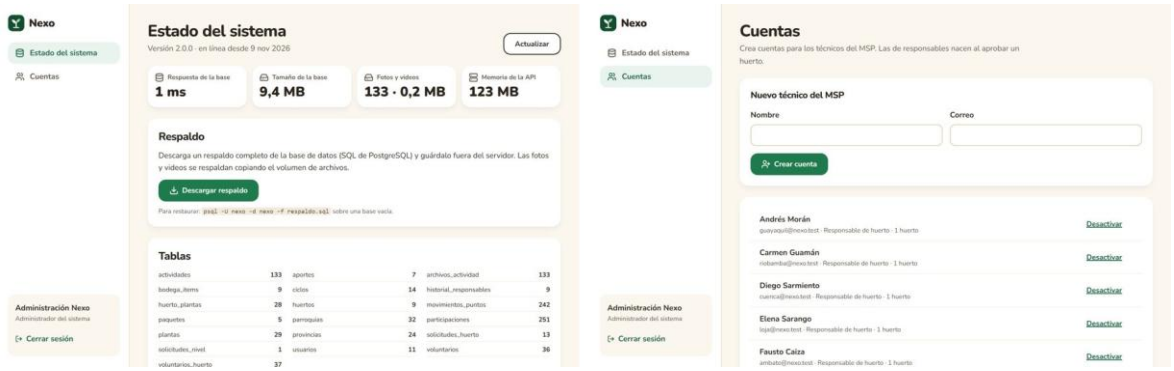


Figura 7.20. Administrador: estado del sistema y cuentas

7.6 Evidencia del entorno operativo

La Figura 7.21 muestra los cuatro contenedores en ejecución después de docker compose up --build; la base de datos, el backend y el servidor de correo pasan su verificación de salud.

```
PS C:
exo-huertos-medicinales> docker compose ps
NAME                        IMAGE                       STATUS                    PORTS
nexo-huertos-nexo-backend-1   nexo-huertos-nexo-backend   Up About a minute (healthy)   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
nexo-huertos-nexo-db-1        postgres:16-alpine          Up About a minute (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
nexo-huertos-nexo-frontend-1  nexo-huertos-nexo-frontend  Up About a minute             0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
nexo-huertos-nexo-mail-1      axllent/mailpit:v1.27       Up About a minute (healthy)   0.0.0.0:1025->1025/tcp, [::]:1025->1025/tcp, 0.0.0.0:8025->8025/tcp, [::]:8025->8025/tcp
```

Figura 7.21. Contenedores del sistema en ejecución

8. Conclusiones y trabajo futuro

El producto cumple el alcance completo del proyecto y se puede usar de principio a fin. Una persona de la comunidad solicita su huerto, el técnico del MSP lo aprueba, el responsable recibe su acceso por correo, registra a sus voluntarios y sus plantas, y anota cada paso del cultivo con fotos. Cuando el MSP valida ese trabajo, cada voluntario gana puntos por sus horas, sube de nivel, el huerto avanza en el ranking y todos pueden verlo en la parte pública.

El cambio más importante de esta versión vino de la reunión con la cliente. El modelo de la primera versión, en el que cualquiera creaba su cuenta y su comunidad, no era lo que el MSP necesitaba. Adoptar el nuevo modelo obligó a cambiar las cuentas, el catálogo de plantas y el sistema de puntos, pero las reglas compartidas, las pruebas y la estructura del repositorio permitieron hacerlo sin perder lo que ya funcionaba: los saberes de la luna, las validaciones de cada paso y el huerto virtual se reutilizaron.

La aplicación también enseña. Para cada paso muestra cuándo y cómo hacerlo según los lineamientos del Ministerio, avisa si se siembra fuera de la fase lunar recomendada o si se riega con aspersor, y el calendario lunar del mes está abierto a cualquier persona. Así, quien la usa aprende mientras registra su trabajo.

Como la materia se enfoca en la calidad, cada parte se probó antes de entregarse: 66 pruebas de reglas, 13 de la API, 8 de interfaz con accesibilidad en celular y computador, 20 casos documentados con la plantilla y una auditoría con Lighthouse con resultados entre 96 y 100. En esta versión se encontraron y corrigieron 9 defectos.

Queda como trabajo futuro: cargar el catálogo completo de parroquias del INEC, ayudar al técnico a detectar imágenes hechas con inteligencia artificial, enviar avisos al celular con las actividades recomendadas y publicar la aplicación en un servidor del MSP con un servicio de correo real.

APRENDIZAJE OBTENIDO

Antes de este proyecto, para el equipo un software de calidad era, sobre todo, un software que funcionaba. Ahora entendemos que la calidad se construye desde el principio: escribir historias de usuario con criterios de aceptación claros nos obligó a pensar qué debía pasar cuando algo sale mal, y no solo cuando todo sale bien. Esos mismos criterios se convirtieron después en los casos de prueba.

También aprendimos a entender un tema que no conocíamos. Ninguno de nosotros sabía cómo se trabaja un huerto medicinal ni qué papel tienen la luna y los rituales en la medicina ancestral. Leer los lineamientos del MSP y escuchar a la cliente nos mostró que, antes de programar, hay que comprender el mundo de las personas que van a usar la aplicación.

La reunión con el MSP nos enseñó que los requisitos cambian y que eso es normal. Lo que nos permitió adaptarnos rápido fue tener las reglas separadas y probadas, y llevar una tabla que une cada acuerdo con su historia de usuario y su prueba. Cuando algo quedaba sin definir, aprendimos a proponer una solución justificada en lugar de esperar.

Probar fue otro aprendizaje importante. Las pruebas automáticas encontraron errores que a simple vista no se notaban, incluso una prueba que fallaba solo a veces por un dato mal generado. Medir con Lighthouse nos mostró un problema de rendimiento en el celular que en la computadora no se veía. Registrar cada defecto, corregirlo y volver a medir nos dio una forma ordenada de mejorar.

Por último, aprendimos a trabajar como un equipo de desarrollo real: repartir roles, usar un solo repositorio, revisar los cambios antes de unirlos y llevar una bitácora del avance. Usar IA generativa como apoyo nos ahorró tiempo, pero también nos enseñó que todo lo que produce se debe revisar y validar con las fuentes antes de usarlo.

REFERENCIAS

[1] Ministerio de Salud Pública del Ecuador, Implementación de huertos medicinales en los establecimientos de salud del primer nivel de atención. Lineamientos, 1.ª ed. Quito, Ecuador: Dirección Nacional de Salud Intercultural – MSP, 2017.

[2] Microsoft, «TypeScript». [En línea]. Disponible en: https://www.typescriptlang.org (consultado el 1 de noviembre de 2026).

[3] «NestJS: a progressive Node.js framework». [En línea]. Disponible en: https://nestjs.com (consultado el 1 de noviembre de 2026).

[4] Meta, «React». [En línea]. Disponible en: https://react.dev (consultado el 1 de noviembre de 2026).

[5] Tailwind Labs, «Tailwind CSS». [En línea]. Disponible en: https://tailwindcss.com (consultado el 1 de noviembre de 2026).

[6] PostgreSQL Global Development Group, «PostgreSQL». [En línea]. Disponible en: https://www.postgresql.org (consultado el 1 de noviembre de 2026).

[7] Prisma, «Prisma ORM». [En línea]. Disponible en: https://www.prisma.io (consultado el 1 de noviembre de 2026).

[8] «pnpm». [En línea]. Disponible en: https://pnpm.io (consultado el 1 de noviembre de 2026).

[9] Docker Inc., «Docker Docs». [En línea]. Disponible en: https://docs.docker.com (consultado el 1 de noviembre de 2026).

[10] Anthropic, «Claude», asistente de IA generativa. [En línea]. Disponible en: https://claude.ai (consultado el 1 de noviembre de 2026).

[11] Leaflet y OpenStreetMap, «Leaflet: an open-source JavaScript library for interactive maps». [En línea]. Disponible en: https://leafletjs.com (consultado el 9 de noviembre de 2026).

[12] Axllent, «Mailpit: email and SMTP testing tool». [En línea]. Disponible en: https://mailpit.axllent.org (consultado el 9 de noviembre de 2026).

[13] Microsoft, «Playwright». [En línea]. Disponible en: https://playwright.dev (consultado el 9 de noviembre de 2026).

[14] Deque Systems, «axe-core: accessibility engine for automated web UI testing». [En línea]. Disponible en: https://github.com/dequelabs/axe-core (consultado el 9 de noviembre de 2026).

[15] Google, «Lighthouse». [En línea]. Disponible en: https://developer.chrome.com/docs/lighthouse (consultado el 9 de noviembre de 2026).

Uso de IA generativa: se utilizó Claude (Anthropic) como apoyo en la planificación del proyecto, la propuesta del esquema de datos, el desarrollo y la revisión del código, las pruebas y la organización de este informe. El equipo revisó y validó el contenido producido.